DAY 8 | HANDS-ON ACTIVITIES

JavaScript and Async JavaScript: Problem Statements

Seven problems: three on JavaScript fundamentals and callbacks, four on async JavaScript. Run JavaScript with Node.js (in Colab: !node file.js).

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: JavaScript Warm-up

Level: Warm-up | **Concepts:** const, let, functions, template literals

Write functions to convert Celsius to Fahrenheit, check if a number is even, and build a greeting with a template literal. Print the results with console.log.

Sample run / expected result

```
37 C = 98.6 F
7 is odd
Hello, Aiko! Welcome to Day 8.
```

Hints

- const toF = (c) => c * 9 / 5 + 32;

Problem 2: Ticket Array Workout

Level: Easy | **Concepts:** map, filter, reduce, objects

Given an array of SmartDesk ticket objects (id, title, priority, ageHours, status), print the titles of open high-priority tickets, the average age of open tickets, and the oldest ticket.

Sample run / expected result

```
High and open: No signal at home, Double charge
Average open age: 13.7 h
Oldest: T-104 (30 h)
```

Hints

- reduce((max, t) => t.ageHours > max.ageHours ? t : max)

Problem 3: Group By with a Callback

Level: Medium | **Concepts:** callbacks, objects as maps

Write groupBy(items, keyFn) where keyFn is a callback that returns the group name. Use it to group tickets by status, then by "old" or "new" (older than 24 hours).

Sample run / expected result

```
{ open: 4, pending: 2, closed: 1 }
{ old: 2, new: 5 }
```

Hints

- Start with {} and push into acc[keyFn(item)] ??= [].

Problem 4: Predict the Order

Level: Easy | **Concepts:** event loop, setTimeout

Write a script with three console.log calls and two setTimeout calls (0 ms and 100 ms) mixed in. Predict the output on paper, then run it and compare.

Sample run / expected result

```
A
C
B (timeout 0)
D (timeout 100)
```

Hints

- Synchronous code always finishes before any timer callback.

Problem 5: Promise Kitchen

Level: Medium | **Concepts:** new Promise, then, catch

Write cook(dish, ms) that resolves after ms milliseconds, or rejects if dish is "pizza" (out of cheese). Chain three dishes with then and handle the error with catch.

Sample run / expected result

```
rice ready
curry ready
Error: out of cheese for pizza
```

Hints

- return new Promise((resolve, reject) => setTimeout(...))

Problem 6: Sequential vs Parallel

Level: Medium | **Concepts:** async/await, Promise.all, timing

Cook three dishes of 500 ms each sequentially with await, then in parallel with Promise.all. Print how long each approach took.

Sample run / expected result

```
Sequential: 1503 ms
Parallel: 502 ms
```

Hints

- const start = Date.now(); ... Date.now() - start

Problem 7: Async Ticket Client

Level: Stretch | **Concepts:** fetch, JSON, async/await, error handling

Using the small ticket server from the notebook, write client.js that fetches /tickets, prints a count per priority, and fetches /tickets/999 to show a clean error for a 404. Use JSON.stringify to print one ticket.

Sample run / expected result

```
Loaded 6 tickets
{ high: 2, medium: 3, low: 1 }
Error: HTTP 404 for /tickets/999
```

Hints

- if (!res.ok) throw new Error(`HTTP ${res.status}`)
- Wrap the whole flow in try / catch.